INFORME TÉCNICO: PRIMERA ENTREGA (E1)

Curso: Arquitectura de Computadoras

Proyecto: Procesador RISC-V Superescalar Dual-Issue

GitHub: https://github.com/jose-lam/riscv-dual-issue-superscalar.git

Integrantes:

1. Sebastian Romero Pahuara - 202220338
2. Jefri Jacson Vargas Fernandez - 202220354
3. Francesco Aroldo Ferrante Quino - 202510174
4. José Enrique Hilario Ruiz Lam - 202510050

1. Arquitectura Pipeline Single-Issue y Unidad de Control

1.1 Descripción del Pipeline de 5 Etapas

- **Fetch (IF):**

  El registro de contador de programa (PCF) envía la dirección actual a la memoria de instrucciones (`imem`) para extraer la instrucción de 32 bits (`InstrF`). En paralelo, un sumador calcula la siguiente dirección secuencial (`PCPlus4F = PCF + 4`). El multiplexor de entrada del PC selecciona entre `PCPlus4F` y la dirección de salto `PCTargetE` (controlado por `PCSrcE`). Al final del ciclo, `InstrF`, `PCF` y `PCPlus4F` se almacenan en el registro de segmentación IF/ID.

- **Decode (ID):**

  Recibe la instrucción `InstrD` desde el registro IF/ID. Envía los campos op (`InstrD[6:0]`), `funct3` (`InstrD[14:12]`) y `funct7b5` (`InstrD[30]`) a la Unidad de Control para generar las señales de control de la etapa (`RegWriteD`, `ResultSrcD`, `MemWriteD`, `JumpD`, `BranchD`, `ALUControlD`, `ALUSrcD`, `ImmSrcD`). Simultáneamente, lee los operandos fuente RD1 (`rs1 = InstrD[19:15]`) y RD2 (`rs2 = InstrD[24:20]`) en el Register File, y extiende el valor inmediato a 32 bits (`ImmExtD`) en la unidad `Extend`. Todos los datos, junto con el identificador del

registro destino RdD (InstrD[11:7]), se registran en el banco ID/EX.

- **Execute (EX):**

  Opera sobre los valores almacenados en el registro ID/EX. La ALU ejecuta la operación aritmética o lógica indicada por ALUControlE entre el primer operando SrcAE y el segundo operando SrcBE (seleccionado mediante el multiplexor ALUSrcE entre el registro leído WriteDataE y el inmediato ImmExtE). En paralelo, un sumador dedicado calcula la dirección destino de salto (PCTargetE = PCE + ImmExtE) y se evalúa la condición de salto combinando BranchE, la bandera ZeroE de la ALU y JumpE para generar PCSrcE. El resultado ALUResultE, el dato a escribir en memoria WriteDataE, RdE y PCPlus4E avanzan al registro EX/MEM.

- **Memory (MEM):**

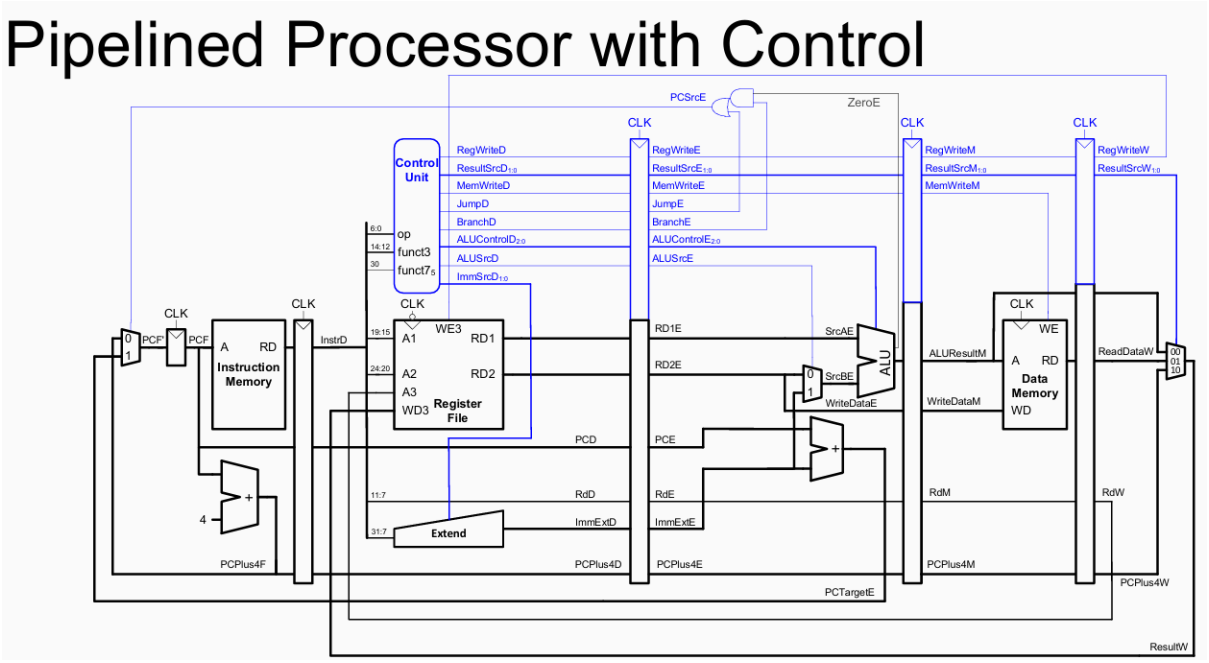
  Utiliza las señales del registro EX/MEM para acceder a la memoria de datos (dmem). El resultado de la ALU (ALUResultM) actúa como dirección de memoria. Si la instrucción es un *store* (MemWriteM = 1), escribe WriteDataM en memoria; si es un *load*, lee el dato ReadDataM. Al finalizar el ciclo, ALUResultM, ReadDataM, RdM y PCPlus4M se guardan en el registro MEM/WB.

- **Writeback (WB):**

  Extrae las señales del registro MEM/WB. Un multiplexor de 3 entradas controlado por ResultSrcW selecciona el resultado final ResultW entre ALUResultW (00), ReadDataW (01) y PCPlus4W (10). Si RegWriteW = 1, escribe ResultW en el registro destino RdW dentro del Register File durante el flanco de bajada del reloj (negedge clk), permitiendo que otra instrucción en Decode pueda leerlo en la segunda mitad del mismo ciclo.

1.2 Datapath Single-Issue y Unidad de Control

- **Diagrama del Datapath Single-Issue:**



- **Unidad de Control:**

  La Unidad de Control es responsable de decodificar las instrucciones y generar las señales necesarias para controlar el funcionamiento del datapath.

  Las principales señales de control son:

  - RegWrite: Habilita la escritura de resultados en el Register File.
  - MemWrite: Habilita la escritura en la memoria de datos para instrucciones sw.
  - ALUSrc: Selecciona el segundo operando de la ALU entre un registro y un valor inmediato.
  - ALUControl: Determina la operación que ejecutará la ALU, como suma, resta, AND, OR o SLT.
  - ResultSrc: Selecciona el resultado que se escribirá en el Register File: resultado de la ALU (00), dato leído de memoria (01) o PC + 4 (10).
  - ImmSrc: Determina el formato del inmediato que debe generar la unidad Extend según el tipo de instrucción.
  - Branch y Jump: Identifican instrucciones de salto condicional e incondicional, respectivamente.

  Estas señales se generan inicialmente en Decode (ID) y las que corresponden avanzan por los registros del pipeline hasta las etapas donde son necesarias. En Execute (EX), las señales BranchE, JumpE y ZeroE permiten determinar PCSrcE, que controla la selección de la siguiente dirección del PC.

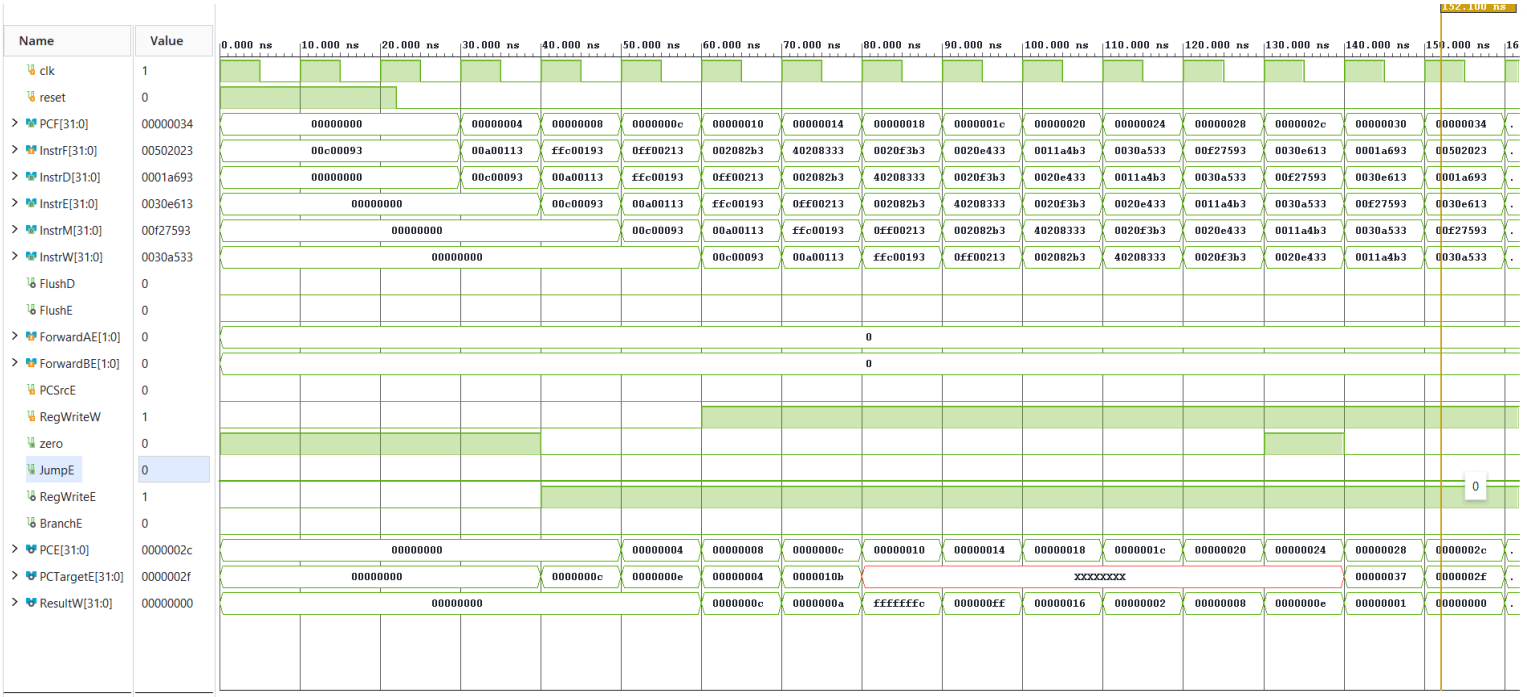
1.3 Validación funcional: Programa 1 (ISA sin dependencias)

- **Código Assembly y .dat:**

None

```
00c00093 // 00: addi x1,x0,12
00a00113 // 04: addi x2,x0,10
ffc00193 // 08: addi x3,x0,-4
0ff00213 // 0c: addi x4,x0,255
002082b3 // 10: add  x5,x1,x2
40208333 // 14: sub  x6,x1,x2
0020f3b3 // 18: and  x7,x1,x2
0020e433 // 1c: or   x8,x1,x2
0011a4b3 // 20: slt  x9,x3,x1
0030a533 // 24: slt  x10,x1,x3
00f27593 // 28: andi x11,x4,15
0030e613 // 2c: ori  x12,x1,3
0001a693 // 30: slti x13,x3,0
00502023 // 34: sw   x5,0(x0)
00602223 // 38: sw   x6,4(x0)
00002783 // 3c: lw   x15,0(x0)
00402803 // 40: lw   x16,4(x0)
00208463 // 44: beq  x1,x2,+8
00100893 // 48: addi x17,x0,1
01078933 // 4c: add  x18,x15,x16
```

- **Waveform de Simulación:**



- **Análisis de Resultados Intermedios:**

En el waveform se verifica el avance continuo de las instrucciones por las cinco etapas (InstrF → InstrW) sin intervención de la Hazard Unit (ForwardAE = 0, ForwardBE = 0, FlushD = 0, FlushE = 0):

➔ Inicialización (0x00–0x0C): Tras desactivar el reset (20 ns), ingresan las cuatro instrucciones addi. A partir de 60 ns, con RegWriteW = 1, se escriben secuencialmente en ResultW los valores 0x0c ($x1 = 12$), 0x0a ($x2 = 10$), 0xfffffffc ($x3 = -4$) y 0xff ($x4 = 255$).

➔ Operaciones ALU (0x10–0x30): Gracias a la escritura del Register File en flanco de bajada (negedge clk), add x5, x1, x2 lee los registros actualizados directamente en Decode sin requerir forwarding. Entre 100 ns y 180 ns, ResultW refleja los resultados de add (0x16), sub (0x02), and (0x08), or (0x0e), slt (0x01, 0x00), andi (0x0f), ori (0x0f) y slti (0x01).

➔ Memoria y Control (0x34–0x4c): Las instrucciones sw guardan $x5$ (22) y $x6$ $(2$) en memoria, lw los recupera en $x15$ y $x16$, y se ejecuta el beq no tomado (PCSrcE = 0) seguido de la suma final $x18 = 22 + 2 = 24$ (0x18).

2. Hazard Unit y Mecanismo de Forwarding

2.1 Diseño de la Hazard Unit para Forwarding

- **Lógica de Detección:**
  El mecanismo de forwarding permite resolver dependencias de datos RAW (Read After Write) sin detener el pipeline, redirigiendo los resultados de instrucciones anteriores hacia la etapa Execute (EX), antes de que sean escritos en el Register File.

  Para activar el forwarding deben cumplirse las siguientes condiciones:

  - Forwarding desde MEM: El registro fuente en EX coincide con RdM, RegWriteM = 1 y el registro fuente es distinto de x0.
  - Forwarding desde WB: El registro fuente en EX coincide con RdW, RegWriteW = 1 y el registro fuente es distinto de x0.

  La señal RegWrite garantiza que la instrucción anterior realmente escribirá un resultado en el Register File, mientras que la comparación con x0 evita dependencias falsas.

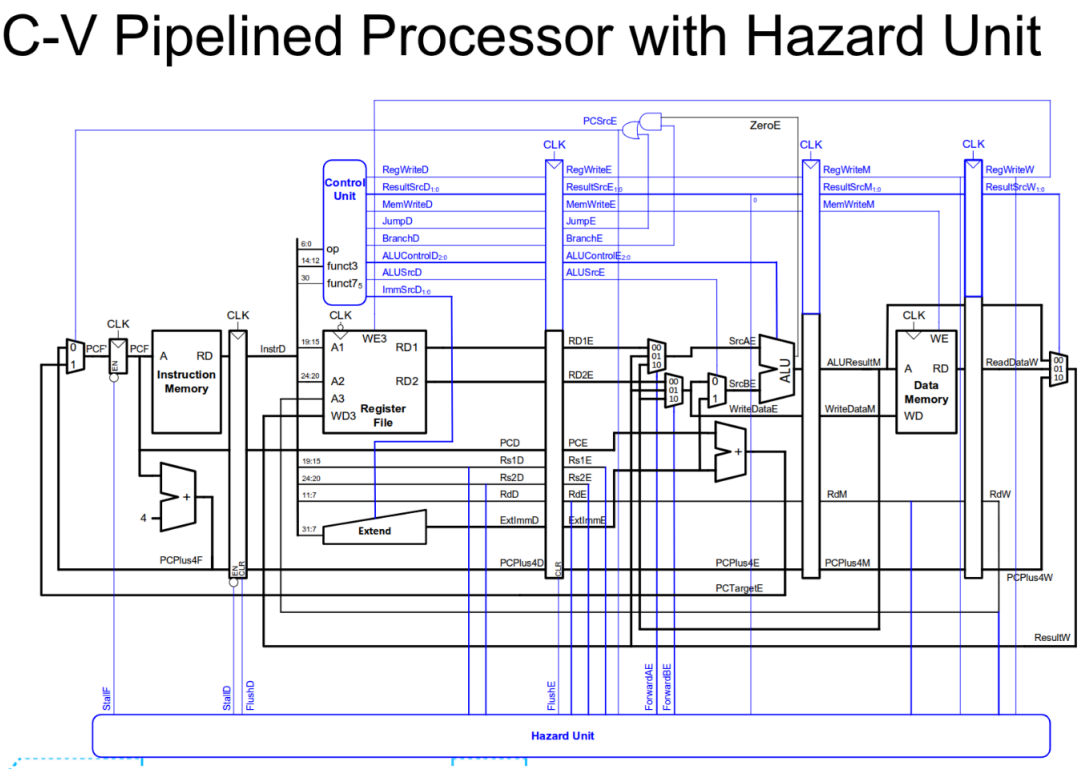
- **Multiplexores de Bypass:**
  Para implementar el forwarding, se incorporan dos multiplexores de tres entradas controlados por las señales ForwardAE y ForwardBE, que seleccionan los operandos utilizados en la etapa Execute.

Cada multiplexor permite seleccionar entre tres fuentes:

Señal	Valor	Fuente seleccionada
ForwardAE	00	RD1E
ForwardAE	01	ResultW
ForwardAE	10	ALUResultM
ForwardBE	00	RD2E
ForwardBE	01	ResultW
ForwardBE	10	ALUResultM

De esta manera, los resultados pueden enviarse directamente desde los registros EX/MEM o MEM/WB hacia Execute, evitando esperar a que sean escritos en el Register File y permitiendo resolver dependencias RAW sin introducir ciclos de stall, siempre que el dato requerido ya esté disponible.

- **Diagrama del Datapath Pipeline Single-Issue con Hazard Unit:**

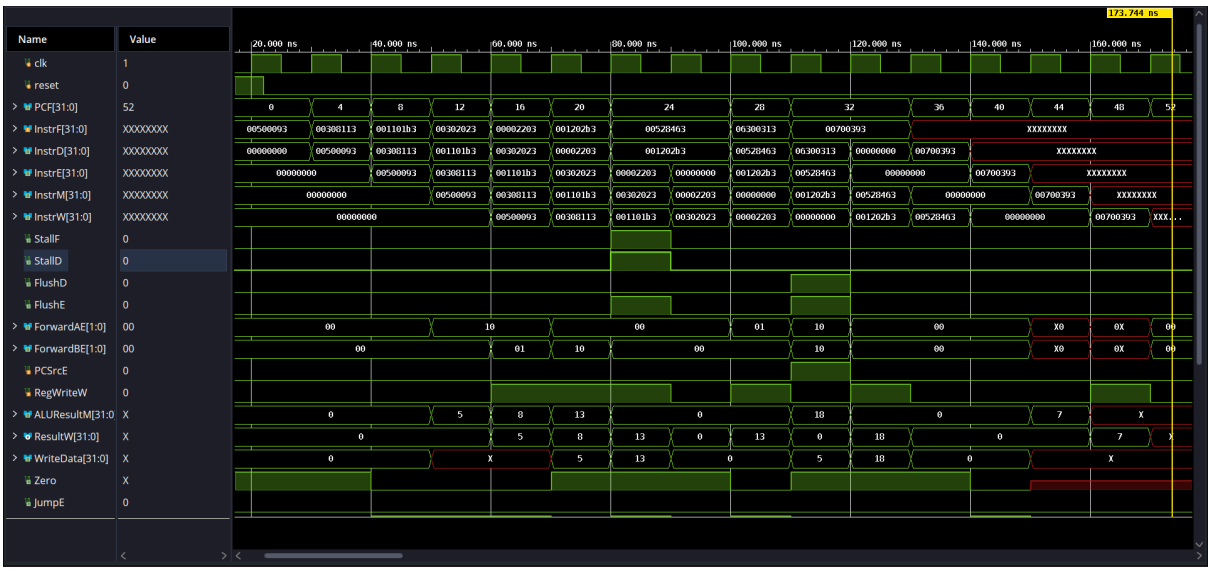


2.2 Validación funcional: Programa 2 (Forwarding)

- **Código Assembly y .dat:**

```
None
00500093 // 00: addi x1,x0,5
00308113 // 04: addi x2,x1,3
001101b3 // 08: add  x3,x2,x1
00302023 // 0c: sw   x3,0(x0)
00002203 // 10: lw   x4,0(x0)
001202b3 // 14: add  x5,x4,x1
00528463 // 18: beq  x5,x5,+8
06300313 // 1c: addi x6,x0,99
00700393 // 20: addi x7,x0,7
```

- **Waveform de Simulación:**



- **Análisis de Señales y Resultados:**

En la simulación, ForwardAE y ForwardBE cambian de 00 a 10 o 01 cuando una instrucción dependiente entra en EX, permitiendo que los resultados disponibles lleguen directamente a la ALU sin introducir stalls innecesarios. Entre 50 y 60 ns, `addi x2,x1,3` recibe el valor de x1 desde ALUResultM (ForwardAE = 10) y produce 8. Entre 60 y 70 ns, `add x3,x2,x1` toma x2 desde ALUResultM (ForwardAE = 10) y x1 desde ResultW (ForwardBE = 01) en el mismo ciclo, obteniendo 8 + 5 = 13. Entre 70 y 80 ns, `sw x3,0(x0)` redirige el dato que se almacenará en memoria (ForwardBE = 10), por lo que WriteDataM vale 13 en el ciclo siguiente. Durante estos tres ciclos, InstrE avanza normalmente y StallF, StallD y FlushE permanecen en 0, mientras ALUResultM muestra los valores 5, 8 y 13 en ciclos consecutivos.

Entre 100 y 120 ns, el forwarding permite que `add x5,x4,x1` reciba desde ResultW el valor 13 cargado previamente por `lw`. La ALU suma este valor con x1 = 5, obteniendo x5 = 18. Posteriormente, `beq x5,x5,+8` recibe el valor actualizado de x5 desde ALUResultM,

permitiendo que la comparación active la señal Zero y se tome el salto. La única detención por dependencia de datos ocurre entre 80 y 90 ns, debido a que el dato de `lw` se obtiene durante MEM y todavía no está disponible para la instrucción dependiente en EX. Por ello, la Hazard Unit introduce un stall de un ciclo, tras el cual ForwardAE = 01 permite resolver la dependencia.

3. Mecanismo de Stalling (Hazard Control)

3.1 Manejo de Riesgos Load-Use

- **Condición de Stall:**
  Las dependencias Load-Use no pueden resolverse únicamente mediante forwarding, debido a que el dato de memoria aún no está disponible cuando la instrucción dependiente necesita ejecutarse en la etapa Execute (EX).

- **Señales de Retención e Inyección de Burbujas:**
  La Hazard Unit activa las señales StallF, StallD y FlushE para detener temporalmente el avance de las instrucciones dependientes.
  - StallF (PC Write): Impide la actualización del Program Counter (PC), manteniendo la misma dirección de instrucción durante un ciclo de reloj.
  - StallD (IF/ID Write): Impide la actualización del registro IF/ID, conservando la instrucción dependiente en la etapa Decode (ID).
  - FlushE (ID/EX Clear): Limpia el registro ID/EX, colocando sus señales de control en cero e insertando una burbuja (NOP) en Execute (EX). Esto evita que la instrucción dependiente se ejecute antes de que el dato esté disponible.

  Estas señales se definen mediante las siguientes expresiones:

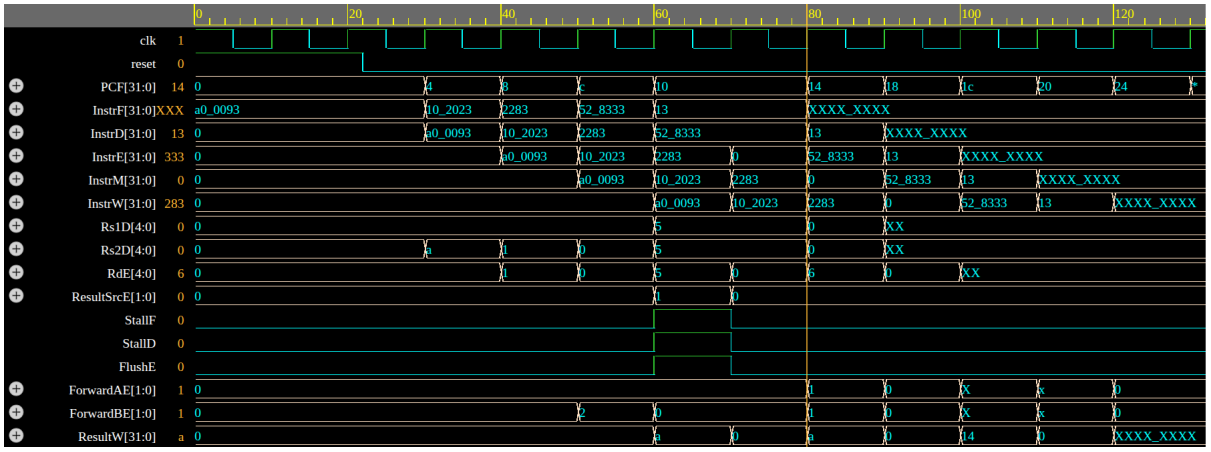
  - StallF = lwStall
  - StallD = lwStall
  - FlushE = lwStall or PCSrcE

3.2 Validación funcional: Programa 3 (Stalling)

- **Código Assembly y .dat:**

```
None
00a00093 // 00: addi x1, x0, 10 (x1 = 10)
00102023 // 04: sw x1, 0(x0) (Mem[0] = 10)
00002283 // 08: lw x5, 0(x0) (x5 = 10)
00528333 // 0c: add x6, x5, x5 (LOAD-USE -> STALL, x6 = 20)
00000013 // 10: nop
```

- **Waveform de Simulación:**



- **Análisis del Comportamiento:**

  La simulación permite observar cómo la Hazard Unit detecta y resuelve una dependencia Load-Use entre las instrucciones lw x5, 0(x0) y add x6, x5, x5:

  - La instrucción lw se encuentra en Execute (EX), mientras que add está en Decode (ID). Como Rs1D = Rs2D = RdE = 5 y ResultSrcE = 01, se activa lwStall, generando StallF = 1, StallD = 1 y FlushE = 1.
  - El PC permanece en 0x10 y la instrucción add continúa en Decode. Mientras tanto, lw avanza a Memory (MEM) y se inserta una burbuja en Execute (InstrE = 0x00000000). Al desaparecer la condición de riesgo en EX, las señales de stall se desactivan.
  - La instrucción lw llega a Writeback (WB) y add avanza a Execute. Se activan ForwardAE = 01 y ForwardBE = 01, permitiendo que ambos operandos de la ALU reciban ResultW = 0x0000000A (10 en decimal).
  - La instrucción add calcula correctamente 10 + 10 = 20, cuyo resultado (0x00000014) se observa posteriormente en Writeback.

4. Flushing e Inicio de la Arquitectura Superescalar Dual-Issue

4.1 Manejo de Riesgos de Control (Flushing)

- **Detección de Branches y Anulación:**

  En la etapa de Execute (E), la unidad de control evalúa la señal de salto junto con el resultado de la ALU (flag Zero) para determinar la activación de PCSrcE.

  - Si no se activa (PCSrcE = 0), el salto no es tomado y las instrucciones que se ejecutan consecutivamente son válidas.

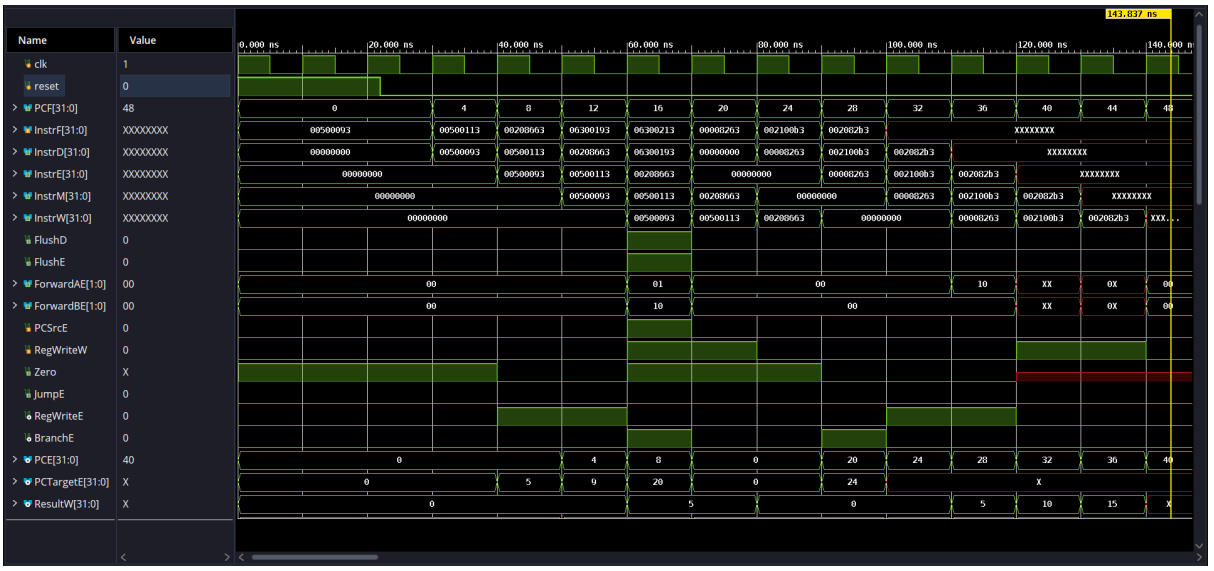
- Si se activa (PCSrcE = 1), el salto es tomado y las instrucciones que entraron en el pipeline por predicción no tomada son erróneas, por lo que deben ser eliminadas (flushed).

La Hazard Unit utiliza PCSrcE para activar las señales FlushD y FlushE. Estas se encargan de limpiar los registros de segmentación Decode y Execute introduciendo burbujas (instrucciones NOP / 0x00000000). Simultáneamente, el multiplexor del PC selecciona PCTargetE hacia PCNext, permitiendo retomar la ejecución desde la dirección del salto.

- **Programa 4 (Flushing):**

```
None
00500093 // 00: addi x1, x0, 5    (x1 = 5)
00500113 // 04: addi x2, x0, 5    (x2 = 5)
00208663 // 08: beq  x1, x2, +12  (SALTO TOMADO -> genera FLUSH en 0x0C y 0x10)
06300193 // 0c: addi x3, x0, 99   (Víctima 1 - Eliminada por Flush)
06300213 // 10: addi x4, x0, 99   (Víctima 2 - Eliminada por Flush)
00008263 // 14: beq  x1, x0, +4   (SALTO NO TOMADO: 5 != 0 -> SIN Flush, flujo
normal)
002100b3 // 18: add  x1, x2, x2   (x1 = 5 + 5 = 10)
002082b3 // 1c: add  x5, x1, x2   (x5 = 10 + 5 = 15)
```

- **Waveform de Flushing:**



- **Análisis:**

En la simulación se observa un comportamiento interesante cuando la instrucción de salto 00208663 (beq x1, x2, +12) llega a la etapa InstrE:

- Debido a que los registros x1 y x2 fueron modificados por las dos instrucciones inmediatas anteriores, la Hazard Unit lo detecta y activa ForwardAE = 01 (para reenviar x1 desde la etapa Writeback / ResultW) y ForwardBE = 10 (para reenviar x2 desde la etapa Memory / ALUResultM).
- Al ser ambos valores iguales a 5, la bandera Zero se pone en nivel alto y activa la señal PCSrcE = 1. Esto escribe la dirección de destino PCTargetE (0x14 / 20) en PCNext para el siguiente ciclo de Fetch.
- Al activarse PCSrcE, la Hazard Unit fuerza a nivel alto FlushD y FlushE. Esto descarta las instrucciones 06300193 (en Decode) y 06300213 (en Fetch), convirtiéndolas en NOPs.
- En los ciclos posteriores, el procesador retoma la ejecución normal directamente desde la instrucción reubicada en 0x14 (00008263).

4.2 Incorporación Inicial Dual-Issue (Dual Fetch & Dual Decode)

- **Modificaciones en Fetch (Dual Fetch):**

  La memoria de instrucciones (imem.v) pasó de entregar una palabra de 32 bits a entregar 64 bits por ciclo. A partir de la dirección del PC devuelve dos instrucciones consecutivas: Instr0 (dirección PC) en [31:0] e Instr1 (dirección PC+4) en [63:32]. Como se indexa por palabra, la lectura funciona desde cualquier dirección y no exige que el PC esté alineado a 8 bytes.

  El procesador mantiene un solo registro de PC. En flujo secuencial avanza de 8 en 8, porque se consumen dos instrucciones por ciclo. Se calculan dos sumas a partir del PC:

  - PC+4: es el PC de la instrucción 1 y la dirección de enlace de la vía 0.
  - PC+8: es el siguiente PC secuencial y la dirección de enlace de la vía 1.

- **Modificaciones en Decode (Dual Decode):**

  Se instanciaron dos veces el módulo controller (c_0 y c_1), sin duplicar el código de maindec ni de aludec. Cada controlador recibe los campos op, funct3 y funct7[5] de su instrucción: InstrD[6:0], [14:12] y [30] para la vía 0, e InstrD[38:32], [46:44] y [62] para la vía 1. Ambos reciben el FlushE del hazard unit, de modo que sus señales de control se anulan junto con el pipeline.

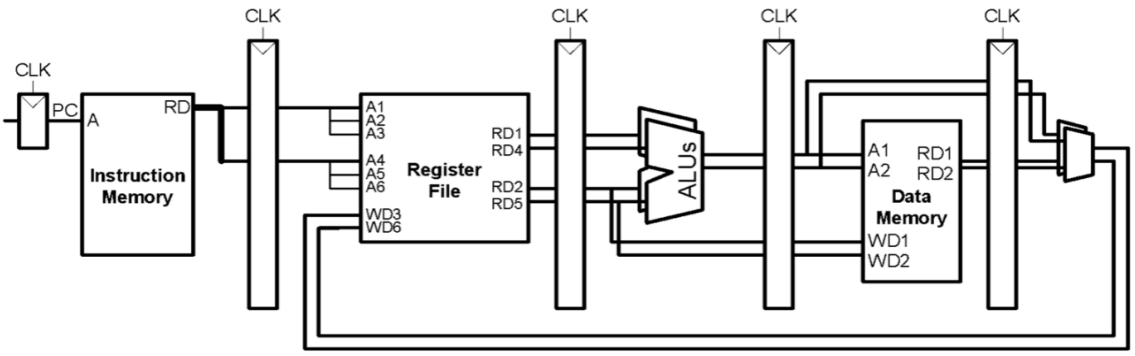
  Se incorporaron los módulos de la segunda vía:

  - Un segundo extensor de inmediato (ext1).
  - Una segunda ALU (alu_1) con sus multiplexores de operandos.
  - Un segundo sumador de dirección de salto.
  - Un segundo multiplexor de resultado en writeback.
  - Los registros de pipeline D/E, E/M y M/W duplicados por vía.

  El Register File pasó a tener 4 puertos de lectura y 2 de escritura. Los puertos a1 y a2 leen los operandos de la vía 0, y a4 y a5 los de la vía 1. Las escrituras de ambas vías se hacen en el flanco de bajada del reloj. Si las dos vías escriben el mismo

registro en un ciclo, prevalece la vía 1, que es la instrucción más joven.

- **Diagrama del Datapath Dual-Issue Preliminar:**



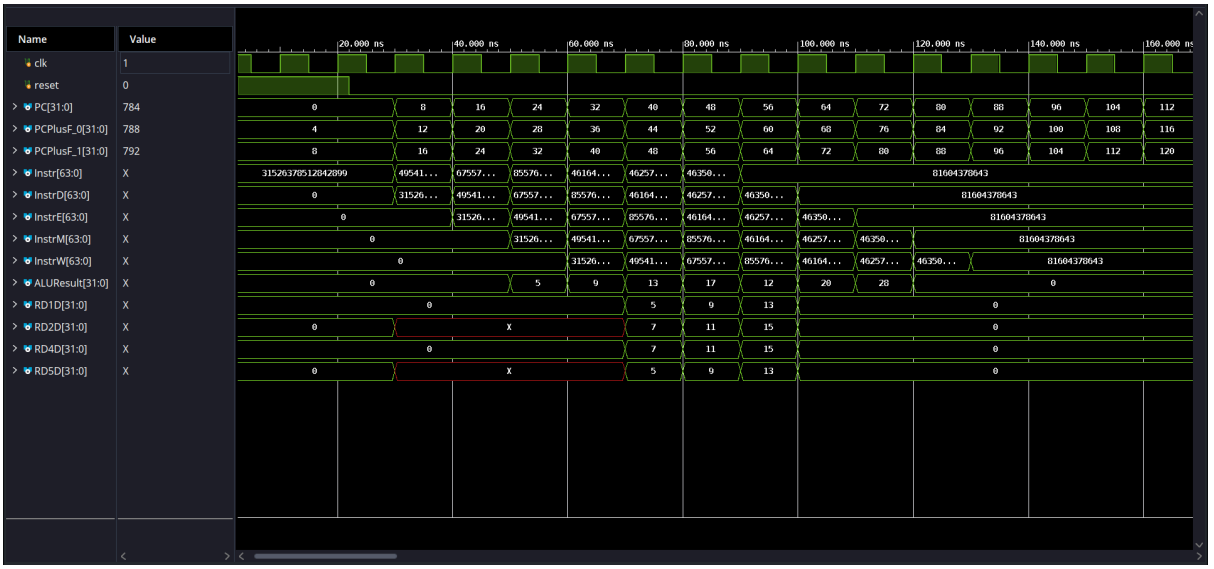
- Un PC, dos sumadores (+4 y +8) y la imem de 64 bits.
- Dos registros F/D.
- Dos controllers, dos extensores y el Register File (4 lecturas, 2 escrituras).
- Dos registros D/E, cada uno con su ALU, sus multiplexores y su sumador de salto.
- Dos registros E/M y dos M/W, con un solo puerto de memoria conectado a la vía 0.
- Dos multiplexores de resultado hacia las 2 escrituras.
- El hazard unit conectado solo a la vía 0.

4.3 Evidencia de Dual Fetch / Dual Decode

- **Waveform de Prueba Dual:**

```
None
00500093 // 00: addi x1,x0,5
00308113 // 04: addi x2,x1,3
001101b3 // 08: add  x3,x2,x1
00302023 // 0c: sw   x3,0(x0)
00002203 // 10: lw   x4,0(x0)
001202b3 // 14: add  x5,x4,x1
00528463 // 18: beq  x5,x5,+8
06300313 // 1c: addi x6,x0,99
00700393 // 20: addi x7,x0,7
00000013 // 24: nop
00000013 // 28: nop
00000013 // 2c: nop
00000013 // 30: nop
00000013 // 34: nop
00000013 // 38: nop
```

```
00000013 // 3c: nop
```



- **Análisis de Señales:**
  El bus Instr[63:0] contiene las dos instrucciones del par, con Instr1 en [31:0] e Instr2 en [63:32]. Con PC = 0 entrega 0x00500093 (addi x1,x0,5) e 0x00700113 (addi x2,x0,7) en el mismo ciclo. Un ciclo después, ambas están en el registro InstrD (30–40 ns) y se decodifican en paralelo: InstrD[31:0] la procesa el controlador c_0 e InstrD[63:32] el controlador c_1. El par avanza unido por InstrE, InstrM e InstrW. La decodificación paralela se ve también en el segundo par (add x3,x1,x2 y sub x4,x2,x1), que en Decode (70–80 ns) provoca cuatro lecturas simultáneas del Register File: RD1D = 5, RD2D = 7 para Instr1, y RD4D = 7, RD5D = 5 para Instr2.